

Lecture 5: The Transport Layer

Course: CS 341 Computer Networks.

This lecture covers the transport layer, the fourth layer of the OSI model and the layer that provides logical communication between application processes running on different hosts. While the network layer moves packets between hosts, the transport layer moves data between processes. The two dominant transport protocols on the Internet are TCP (Transmission Control Protocol) and UDP (User Datagram Protocol).

Learning outcomes: explain multiplexing and demultiplexing with port numbers; compare TCP and UDP; describe the TCP three-way handshake; explain flow control and congestion control; and write a simple socket program.

1. Multiplexing and Port Numbers

A host may run many network applications at once. The transport layer gathers data from several sockets, adds a header, and passes segments to the network layer; this is called multiplexing. At the receiver, demultiplexing delivers each segment to the correct socket.

Sockets are identified using port numbers, which are 16-bit values from 0 to 65535. Ports 0 to 1023 are called well-known ports and are reserved for common services: HTTP uses port 80, HTTPS uses port 443, SSH uses port 22, and DNS uses port 53. Ports 49152 to 65535 are ephemeral ports chosen by the operating system for client connections.

A UDP socket is identified by a two-tuple (destination IP, destination port). A TCP socket is identified by a four-tuple: source IP, source port, destination IP and destination port. This is why a web server can keep thousands of simultaneous TCP connections on port 80.

2. UDP: User Datagram Protocol

UDP is a connectionless, best-effort transport protocol defined in RFC 768. There is no handshake before sending data, no retransmission of lost segments, and no guarantee of ordering. The UDP header is only 8 bytes long and contains four fields: source port, destination port, length and checksum.

Why would anyone use an unreliable protocol? UDP has no connection setup delay, keeps no connection state on the server, has a small header overhead, and is not slowed down by congestion control. These properties make it a good fit for DNS queries, live video and voice streaming, online games and SNMP. Applications that need reliability over UDP must implement it themselves; QUIC, the protocol behind HTTP/3, is a modern example that builds reliable, encrypted streams on top of UDP.

The UDP checksum is computed as the one's complement of the one's complement sum of all 16-bit words in the segment, and it is used to detect bit errors.

3. TCP: Reliable Byte Stream

TCP, defined in RFC 793, is connection-oriented and provides a reliable, in-order byte stream between two processes. It is full-duplex and point-to-point. The minimum TCP header is 20 bytes and includes the sequence number, acknowledgment number, receive window, header length, flags (SYN, ACK, FIN, RST, PSH, URG) and a checksum.

Sequence numbers count bytes, not segments: the sequence number of a segment is the byte-stream number of its first byte. Acknowledgments are cumulative: an ACK number of n means that all bytes up to $n-1$ have been received and the receiver expects byte n next.

TCP uses a single retransmission timer. The timeout interval is computed from an estimated round-trip time: $\text{EstimatedRTT} = (1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$, with α typically 0.125, and $\text{TimeoutInterval} = \text{EstimatedRTT} + 4 * \text{DevRTT}$. A sender also performs a fast retransmit when it receives three duplicate ACKs, without waiting for the timer.

4. Connection Management: the Three-Way Handshake

Before exchanging data, TCP endpoints perform a three-way handshake. Step 1: the client sends a SYN segment with a randomly chosen initial sequence number (client_isn). Step 2: the server allocates buffers and replies with a SYNACK segment carrying its own initial sequence number (server_isn) and acknowledging client_isn + 1. Step 3: the client replies with an ACK acknowledging server_isn + 1; this third segment may already carry application data.

Random initial sequence numbers make it harder for an attacker to inject forged segments. A SYN flood attack sends many SYN segments without completing the handshake, exhausting server resources; SYN cookies are the standard defence.

To close a connection, each side sends a FIN segment and receives an ACK, so closing normally takes four segments. The side that closes first enters the TIME_WAIT state, typically for twice the maximum segment lifetime, before the connection is fully released.

5. Flow Control

Flow control prevents a fast sender from overflowing the receiver's buffer. The receiver advertises the free space in its buffer in the receive window field (rwnd) of every segment it sends. The sender ensures that the amount of unacknowledged data in flight never exceeds rwnd:
 $\text{LastByteSent} - \text{LastByteAcked} \leq \text{rwnd}$.

If the receiver advertises $\text{rwnd} = 0$, the sender stops but continues sending one-byte probe segments so that it learns when space becomes available again. Flow control is a matter between one sender and one receiver; it is different from congestion control, which protects the network itself.

6. Congestion Control

Congestion control limits the sending rate when the network is overloaded. The sender keeps a congestion window (cwnd) and the effective window is $\min(\text{cwnd}, \text{rwnd})$.

TCP Reno uses three phases. Slow start begins with $\text{cwnd} = 1 \text{ MSS}$ and doubles cwnd every RTT, which is exponential growth, until cwnd reaches the slow-start threshold (sssthresh). Congestion avoidance then increases cwnd by 1 MSS per RTT, which is linear growth; this is the additive increase part of AIMD (additive increase, multiplicative decrease). On three duplicate ACKs, Reno halves cwnd and enters fast recovery. On a timeout, sssthresh is set to half of cwnd and cwnd is reset to 1 MSS, returning to slow start.

TCP CUBIC, the default in Linux since kernel 2.6.19, grows the window as a cubic function of the time since the last loss, which works better on high-bandwidth, long-delay paths. BBR, developed by Google, instead models the bottleneck bandwidth and round-trip propagation time.

7. Socket Programming in Python

The socket API is the interface between applications and the transport layer. In Python a TCP server is written as follows: create a socket with `socket(AF_INET, SOCK_STREAM)`, call `bind()` with the server address and port, call `listen()` to wait for clients, and call `accept()`, which returns a new connection socket for each client. The client calls `connect()` and then uses `send()` and `recv()`.

A UDP socket is created with `SOCK_DGRAM` instead of `SOCK_STREAM`. UDP sockets do not call `listen()`, `accept()` or `connect()`; they use `sendto()` and `recvfrom()`, and every datagram carries the destination address explicitly.

ملخص المحاضرة: مقارنة بين TCP و UDP

بروتوكول TCP بروتوكول موجّه بالاتصال، يبدأ بالمصافحة الثلاثية قبل إرسال البيانات، ويضمن وصول البيانات كاملة وبالترتيب الصحيح عن طريق أرقام التسلسل والإقرارات وإعادة الإرسال. كما يوفر TCP التحكم في التدفق والتحكم في الازدحام.

أما بروتوكول UDP فهو بروتوكول عديم الاتصال، لا يضمن وصول البيانات ولا ترتيبها، لكن رأسه صغير جدًا (ثمانية بايتات فقط) ولا يحتاج إلى وقت لإنشاء الاتصال. لذلك يُستخدم UDP في تطبيقات البث المباشر للفيديو والصوت والألعاب عبر الإنترنت واستعلامات DNS.

القاعدة العامة: استخدم TCP عندما تكون الموثوقية أهم من السرعة، واستخدم UDP عندما يكون التأخير المنخفض أهم من فقدان بعض الحزم.